SISTEMAS OPERATIVOS MONOPUESTO (SMR 1.º)

Scripting básico en Bash

Tipo Explicación | **Nivel** 3 · Avanzado | **Tema** 13 | **CM** CM4 | **Duración** 10 h

Herramientas H2 (CMD/PowerShell), H4 (terminal Linux y Bash)

🧭 **Antes de empezar.** Parte 1: variables, condicionales y argumentos. Parte 2: bucles, funciones y automatización de copias de seguridad. El script de copia de seguridad que hagas aquí es una de las piezas del reto ETHAZI.

🧩 **Analogía.** Un script es una nota que le dejas al ordenador: «cuando llegues, haz esto, esto y esto». La diferencia con teclear los comandos a mano es que la nota **no se cansa, no se equivoca y no se olvida de ninguno de los pasos** — pero tampoco piensa. Si la nota está mal escrita, la seguirá igual de fielmente, 500 veces si hace falta.

Índice

- Scripting básico en Bash
  - Índice
  - Parte 1 — Variables, condicionales y argumentos
  - 1. El primer script
  - 2. Ejecutar un script
  - 3. Variables
  - 4. Entrada del usuario: `read`
  - 5. Argumentos
  - 6. Condicionales
    - Comparar números
    - Comparar texto
    - Comprobar ficheros
    - Combinar condiciones
  - 7. Códigos de salida
  - 8. `case`
  - Parte 2 — Bucles, funciones y automatización
  - 9. Bucles
    - `for` sobre una lista
    - `for` sobre ficheros

    - `for` estilo C
    - `while`
    - `while` leyendo un fichero línea a línea
    - `until`, `break` y `continue`
  - 10. Funciones
  - 11. Redirección y registro
  - 12. Depurar un script
  - 13. Programar la ejecución: cron
  - 14. Buenas prácticas

Parte 1 — Variables, condicionales y argumentos

1. El primer script

```
#!/bin/bash

# Esto es un comentario
echo "Hola, mundo"
```

Línea	Qué hace
`#!/bin/bash`	El **shebang**. Le dice al sistema qué intérprete usar. Va **siempre en la primera línea**, sin espacios delante
`#`	Comentario. Todo lo que va detrás se ignora
`echo`	Escribe en pantalla

> ℹ️ **Por qué el shebang importa.** Sin él, si ejecutas el script con `./script.sh` el sistema usa el shell por defecto, que puede no ser Bash. Un script que funciona en tu equipo y falla en el del compañero suele ser esto.

2. Ejecutar un script

Tres formas, y no son equivalentes:

```
bash hola.sh      # 1) el intérprete lee el fichero. No hace falta permiso de ejecución
./hola.sh         # 2) ejecución directa. SÍ hace falta permiso de ejecución
source hola.sh    # 3) se ejecuta en el shell ACTUAL
```

Para la forma 2 hay que dar permiso de ejecución (guía de permisos):

```
chmod +x hola.sh
./hola.sh
```

ℹ️ **La diferencia entre `./script.sh` y `source script.sh`** es la misma que viste al personalizar el terminal con `.bashrc`: `./` abre un shell nuevo, hace su trabajo y se cierra llevándose los cambios; `source` lo ejecuta aquí, así que las variables que defina se quedan. Compruébalo:

```
echo 'MIVAR="hola"' > prueba.sh
./prueba.sh ; echo "[$MIVAR]"      # sale vacío
source prueba.sh ; echo "[$MIVAR]" # sale hola
```

⚠️ **`./` no es opcional.** `hola.sh` a secas no funciona porque el directorio actual **no** está en el `PATH`, y eso es a propósito: evita que un fichero llamado `ls` en una carpeta cualquiera se ejecute en lugar del `ls` real.

3. Variables

```
#!/bin/bash

NOMBRE="Patxi"
EDAD=17

echo "Hola, $NOMBRE"
echo "Tienes $EDAD años"
echo "El año que viene tendrás $((EDAD + 1))"
```

Reglas que ahorran horas de depuración:

```
NOMBRE="Patxi"      # SIN espacios alrededor del =
NOMBRE = "Patxi"    # ERROR: bash cree que 'NOMBRE' es un comando
```

Forma	Qué hace
`$VAR` o `${VAR}`	Valor de la variable
`"$VAR"`	**Entre comillas dobles**: se sustituye la variable. Es lo que quieres casi siempre
`'$VAR'`	Entre comillas simples: **no** se sustituye. Sale literalmente `$VAR`
`$(comando)`	Sustitución de comandos: se sustituye por la salida del comando
`$((operación))`	Aritmética

```
FECHA=$(date +%Y-%m-%d)
USUARIOS=$(who | wc -l)
echo "Hoy es $FECHA y hay $USUARIOS usuarios conectados"
```

> ⚠️ **Pon siempre comillas dobles alrededor de las variables.** Si una variable contiene un espacio y no la entrecomillas, Bash la parte en dos:
>
> ```
> ARCHIVO="mi documento.txt"
> rm $ARCHIVO     # intenta borrar 'mi' y 'documento.txt'  ← MAL
> rm "$ARCHIVO"   # borra el fichero correcto              ← BIEN
> ```
>
> Es el error número uno en scripting y en el entregable se penaliza.

Variables de entorno útiles:

```
echo $HOME $USER $PWD $HOSTNAME $PATH
```

4. Entrada del usuario: `read`

```
#!/bin/bash

echo -n "¿Cómo te llamas? "
read NOMBRE
echo "Hola, $NOMBRE"
```

Más cómodo, con el mensaje incorporado:

```
read -p "¿Cómo te llamas? " NOMBRE
read -sp "Contraseña: " CLAVE   # -s oculta lo que se escribe
echo
```

5. Argumentos

Los datos que se pasan al llamar al script:

```
#!/bin/bash

echo "Nombre del script:  $0"
echo "Primer argumento:   $1"
echo "Segundo argumento:  $2"
echo "Número de argumentos: $#"
echo "Todos:              $@"
```

```
./script.sh manzana pera
```

Variable	Contenido
`$0`	Nombre del script
`$1`, `$2`, `$3`...	Argumentos por posición
`$#`	Cuántos argumentos hay
`$@`	Todos, como elementos separados
`$?`	Código de salida del último comando
`$$`	PID del script

Comprobar que se han recibido los argumentos — esto tiene que estar en todos tus scripts:

```
#!/bin/bash

if [ $# -ne 2 ]; then
    echo "Uso: $0 <origen> <destino>"
    exit 1
fi
```

6. Condicionales

```bash
#!/bin/bash

EDAD=18

if [ "$EDAD" -ge 18 ]; then
    echo "Eres mayor de edad."
else
    echo "Eres menor de edad."
fi
```

> ⚠️ **Los espacios dentro de los corchetes son obligatorios.** `[` no es puntuación: es un comando. `[$EDAD -ge 18]` da error de sintaxis. En los apuntes del curso anterior había justamente ese fallo en el ejemplo del `elif`; escríbelo siempre así: `[ "$EDAD" -ge 18 ]`.

Con varias ramas:

```bash
#!/bin/bash

read -p "Edad: " EDAD

if [ "$EDAD" -ge 80 ]; then
    echo "Eres abuelo."
elif [ "$EDAD" -ge 18 ]; then
    echo "Eres mayor de edad."
else
    echo "Eres menor de edad."
fi
```

Comparar números

Operador	Significa
`-eq`	igual
`-ne`	distinto
`-lt`	menor que
`-le`	menor o igual
`-gt`	mayor que
`-ge`	mayor o igual

Comparar texto

Operador	Significa
`=`	iguales
`!=`	distintos
`-z "$VAR"`	la cadena está vacía
`-n "$VAR"`	la cadena no está vacía

> ⚠️ Los números se comparan con `-eq`, `-lt`... y el texto con `=`, `!=`. Si usas `=` con números, `10 = 010` da falso porque compara los caracteres, no el valor.

Comprobar ficheros

Operador	Verdadero si...
`-e ruta`	existe
`-f ruta`	existe y es un fichero normal
`-d ruta`	existe y es un directorio
`-r ruta`	tengo permiso de **lectura**
`-w ruta`	tengo permiso de **escritura**
`-x ruta`	tengo permiso de **ejecución**
`-s ruta`	existe y no está vacío

Esto conecta directamente con la guía de permisos:

```
#!/bin/bash

FICHERO="$1"

if [ ! -e "$FICHERO" ]; then
    echo "El fichero '$FICHERO' no existe."
    exit 1
fi

[ -r "$FICHERO" ] && echo "  lectura:   sí" || echo "  lectura:   no"
[ -w "$FICHERO" ] && echo "  escritura: sí" || echo "  escritura: no"
[ -x "$FICHERO" ] && echo "  ejecución: sí" || echo "  ejecución: no"
```

Combinar condiciones

```
# Y lógico
if [ "$EDAD" -ge 18 ] && [ "$PAIS" = "ES" ]; then
    echo "Mayor de edad y residente"
fi

# O lógico
if [ "$EDAD" -lt 18 ] || [ "$TUTOR" = "si" ]; then
    echo "Necesita autorización"
fi

# Negación
if [ ! -f "$FICHERO" ]; then
    echo "El fichero no existe"
fi
```

7. Códigos de salida

Todo comando devuelve un número al terminar: **0 significa que fue bien**, cualquier otro número es un error.

```
ls /etc
echo $?          # 0

ls /noexiste
echo $?          # 2

whoam
echo $?          # 127 (comando no encontrado)
```

En tus scripts:

```
exit 0    # todo correcto
exit 1    # error genérico
exit 2    # error de uso (argumentos incorrectos)
```

> ℹ️ **Para qué sirve esto de verdad:** para encadenar. `comando1 && comando2` ejecuta el segundo **sólo si el primero salió bien**. Y una tarea programada (apartado 13) puede avisar de que algo falló mirando ese número.

8. `case`

Para muchas opciones, más legible que un `if` con diez `elif`:

```
#!/bin/bash

read -p "Opción (a/b/c): " OPCION

case "$OPCION" in
    a|A)
        echo "Has elegido A"
        ;;
    b|B)
        echo "Has elegido B"
        ;;
    [0-9])
        echo "Has escrito un número"
        ;;
    *)
        echo "Opción no válida"
        ;;
esac
```

Parte 2 — Bucles, funciones y automatización

9. Bucles

`for` sobre una lista

```
#!/bin/bash

for NUMERO in 1 2 3 4 5; do
    echo "Número: $NUMERO"
done

for FRUTA in manzana pera plátano; do
    echo "Me gusta la $FRUTA"
done
```

`for` sobre ficheros

Es el que más vas a usar:

```
for FICHERO in /etc/*.conf; do
    echo "=== $FICHERO ==="
    head -n 3 "$FICHERO"
done
```

`for` estilo C

```
for (( i=1; i<=10; i++ )); do
    echo "Vuelta $i"
done

for i in {1..10}; do
    echo "Vuelta $i"
done
```

`while`

```
#!/bin/bash

CONTADOR=0

while [ "$CONTADOR" -lt 7 ]; do
    echo "Contador: $CONTADOR"
    ((CONTADOR++))
done
```

`while` leyendo un fichero línea a línea

```
while IFS= read -r LINEA; do
    echo "Línea: $LINEA"
done < /etc/passwd
```

> ℹ️ Esta es la forma correcta de recorrer un fichero. `IFS=` evita que se coman los espacios del principio y `-r` evita que las barras invertidas se interpreten.

`until`, `break` y `continue`

```
until [ -f /tmp/listo ]; do
    echo "Esperando..."
    sleep 2
done
```

```bash
for i in {1..10}; do
    [ "$i" -eq 3 ] && continue   # salta esta vuelta
    [ "$i" -eq 8 ] && break      # sale del bucle
    echo "$i"
done
```

10. Funciones

```bash
#!/bin/bash

saludar() {
    echo "Hola, $1"
}

comprobar_fichero() {
    local RUTA="$1"
    if [ -f "$RUTA" ]; then
        return 0
    else
        return 1
    fi
}

saludar "Patxi"

if comprobar_fichero "/etc/passwd"; then
    echo "El fichero existe"
fi
```

Detalle	Explicación
`local`	La variable sólo existe dentro de la función. **Úsalo siempre** o pisarás variables del script
`$1`, `$2`	Dentro de la función son los argumentos **de la función**, no los del script
`return`	Devuelve un **código de salida** (0-255), no un texto
Devolver texto	Se hace con `echo` y se recoge con `RESULTADO=$(mi_funcion)`

> ⚠️ Las funciones deben definirse **antes** de usarlas. Bash lee el fichero de arriba abajo.

11. Redirección y registro

Símbolo	Qué hace
`>`	Salida normal a un fichero, **sobrescribiendo**
`>>`	Salida normal a un fichero, **añadiendo al final**
`2>`	Salida de **errores** a un fichero
`&>`	Salida normal **y** errores al mismo fichero
`2>&1`	Redirigir los errores a donde vaya la salida normal
`< fichero`	Leer la entrada desde un fichero
`|`	Tubería: la salida de uno es la entrada del siguiente
`/dev/null`	El agujero negro: lo que va ahí se descarta

```
ls /etc > listado.txt
ls /noexiste 2> errores.txt
comando &> todo.log
comando > salida.log 2>&1
comando 2>/dev/null              # ejecutar callando los errores
```

Función de registro (esto va en el proyecto final):

```
LOG="/var/log/mi_copia.log"

registrar() {
    echo "[$(date '+%Y-%m-%d %H:%M:%S')] $1" >> "$LOG"
}

registrar "Comienza la copia de seguridad"
```

12. Depurar un script

```
bash -x script.sh        # muestra cada comando antes de ejecutarlo
bash -n script.sh        # comprueba la sintaxis SIN ejecutar
```

O dentro del script:

```
set -x    # activar el trazado a partir de aquí
set +x    # desactivarlo
```

Y esta línea, al principio de cualquier script serio:

```
set -euo pipefail
```

Opción	Qué hace
`-e`	Sale al primer comando que falle
`-u`	Falla si usas una variable no definida (cazas las erratas en los nombres)
`-o pipefail`	Una tubería falla si falla cualquiera de sus comandos, no sólo el último

> ℹ️ **Por qué importa `-e`.** Sin ella, un script cuyo `cd /destino` falla **sigue ejecutando** las líneas siguientes en el directorio equivocado. Si la línea siguiente es un `rm -rf *`, ya te imaginas.

También hay un comprobador que señala errores típicos:

```
sudo pacman -S shellcheck
shellcheck mi_script.sh
```

13. Programar la ejecución: cron

```
crontab -e     # editar mis tareas programadas
crontab -l     # listarlas
```

Formato de una línea:

```
┌──── minuto (0-59)
│ ┌── hora (0-23)
│ │ ┌ día del mes (1-31)
│ │ │ ┌ mes (1-12)
│ │ │ │ ┌ día de la semana (0-7, 0 y 7 = domingo)
│ │ │ │ │
* * * * *  comando
```

Línea	Cuándo se ejecuta
`0 3 * * *`	Todos los días a las 3:00
`30 22 * * 5`	Los viernes a las 22:30

Línea	Cuándo se ejecuta
`*/15 * * * *`	Cada 15 minutos
`0 2 1 * *`	El día 1 de cada mes a las 2:00

```
0 3 * * * /home/alumno/bin/copia.sh >> /home/alumno/copia.log 2>&1
```

> ⚠️ **Cron ejecuta con un entorno casi vacío.** No tiene tu `PATH` ni tus alias. Usa **siempre rutas absolutas** dentro de los scripts que programes, o funcionarán a mano y fallarán a las tres de la mañana sin que te enteres. Es el error clásico de esta unidad.

El equivalente moderno en systemd son los *timers* (`systemctl list-timers`), y en Windows el **Programador de tareas**.

14. Buenas prácticas

1. Shebang en la primera línea.
2. `set -euo pipefail` en cualquier script que haga algo importante.
3. Comentario de cabecera: qué hace, quién lo escribió, cuándo y cómo se usa.
4. **Comillas dobles en todas las variables.**
5. Comprobar los argumentos antes de trabajar y salir con un mensaje de uso claro.
6. `local` en las variables de las funciones.
7. Rutas absolutas si el script se va a programar.
8. Registrar en un fichero de log lo que hace.
9. Códigos de salida coherentes.
10. **Nunca `rm -rf "$VAR"/` sin comprobar antes que `$VAR` no está vacía.** Si lo está, la orden se convierte en `rm -rf /` .

Plantilla para empezar:

```bash
#!/bin/bash
#============================================================
# Nombre:      copia.sh
# Descripción: Copia de seguridad de una carpeta con rotación
# Autor:       <tu nombre>
# Fecha:       2027-05-xx
# Uso:         ./copia.sh <origen> <destino>
#============================================================

set -euo pipefail

if [ $# -ne 2 ]; then
    echo "Uso: $0 <origen> <destino>" >&2
    exit 2
fi

ORIGEN="$1"
DESTINO="$2"

# ... el resto ...

exit 0
```
